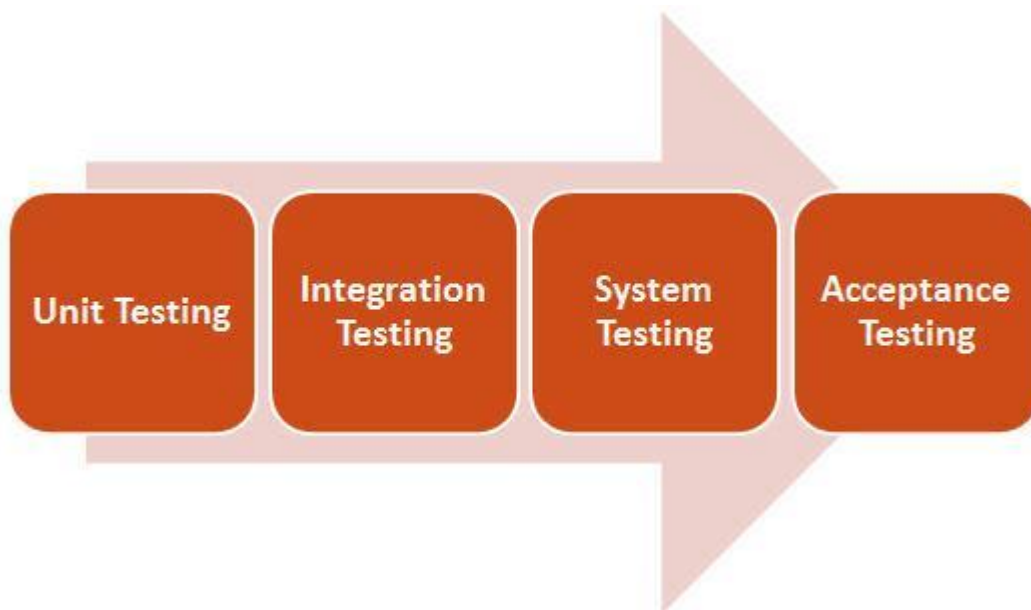


Software testing is a process of evaluating the functionality of a software application with the goal of determining whether the developed software meets specified requirements and identifying errors to ensure that the product is defect-free and of high quality. The stages of software testing are:

- Unit testing
- Intergration testing
- System testing
- Acceptance testing



I. Unit testing

Define:

What is a software unit? A unit is the smallest software component that we can test. It includes functions, procedures, classes, or methods.

Unit testing is performed to verify whether individual modules of source code are functioning correctly. This means testing each unit of the application separately by the developer in the developer environment. This is module testing. Unit testing is a type of white-box testing.

Unit testing is a type of white-box testing.

Purpose:

To ensure that each unit of software is implemented as designed.

Unit testing increases confidence in code changes/maintenance. If good unit tests are written and if they are run every time any code is changed, we will be able to catch any errors caused by the changes in a timely manner.

The cost of fixing bugs discovered during unit testing is lower than that of bugs discovered at a higher level. Compare the cost (time, effort, damage, loss of face) of a bug discovered during acceptance testing or while the software is in operation.

II. Integration Test

Define:

Integration testing is a level of software testing where individual units are combined and tested as a group. A software project consists of many software modules, coded by different people. Integration testing focuses on checking data transmission between modules (integrating functions together, integrating screens together module by module or based on functionality).

Purpose:

To reveal errors in the interaction between integrated units. To find errors in the process of integrating components and modules together. Test drivers and test elements are used to assist in integration testing.

The approaches:

Big Bang: In Big Bang integration testing, individual modules are not integrated until all modules are ready. Then, they are run to check if it works correctly. In this type of testing, some drawbacks can occur, and errors may be found at a later stage. It will be difficult to determine whether the error is in the interface or in the module.

Top Down: In top-down integration testing, high-level modules are integrated and tested first. That is, testing proceeds from the main module down to the sub-modules. In this type of testing, stubs are used as temporary modules if a module is not yet ready for integration testing.

Bottom Up: In Bottom Up integration testing, low-level modules are integrated and tested first, meaning testing proceeds from the sub-module to the main module. Similar to Stubs, the driver here is used as a temporary module for integration testing.

Sandwich/Hybrid is an approach to integration testing that combines Top Down and Bottom Up methods.

III. System Testing – System-level testing

Define:

It is a level of software testing where a complete and integrated software program is tested.

The key difference between Integration Testing and System Testing is that System Testing focuses on the behavior and errors across the entire system, while Integration Testing focuses on the communication between individual units or objects as they work together. Typically, Unit Testing and Integration Testing must be performed to ensure all units and their interactions are functioning correctly before proceeding with System Testing.

System testing includes both functional and non-functional testing.

System testing focuses more on the functionality of the entire system.

System test cases encompass the functionalities of the complete product and involve high-level testing.

The behavior of the complete application is tested to meet the specified requirements.

Test cases and test data are executed, and actual data is not used in this type of testing.

Purpose:

To assess the system's compliance with the specified requirements.

IV. Acceptance Testing

Define:

It is a level of software testing in which a system is tested for acceptance.

Acceptance testing involves testing functionalities to verify system behavior using real-world data. It is also known as enterprise user testing.

Acceptance testing is performed by end users to verify that the system built meets the organization's business requirements.

Purpose:

Assess the system's compliance with business requirements and evaluate whether it is acceptable for distribution.

The goal of acceptance testing is to reaffirm confidence in the system, its functional and non-functional characteristics. Error searching is not the primary focus of

acceptance testing. Acceptance testing can assess the system's readiness for deployment and use, although it is not necessarily the final stage of testing.

Acceptance testing types:

Alpha Testing: is a form of acceptance testing. Alpha testing is performed by a group of people at the software development site. It is a type of internal acceptance testing, preceding beta testing of the software.

Beta testing is a form of testing that follows alpha testing. It is performed at the customer's location, not at the software development site.